


**VILNIAUS UNIVERSITETAS**

**MATEMATIKOS IR INFORMATIKOS FAKULTETAS**

**DUOMENŲ MOKSLO STUDIJŲ PROGRAMA**

Kursinis darbas

# Natūralios kalbos apdorojimo metodų taikymas muitinės deklaracijų prekių klasifikavimo ir neatitikimų aptikimo uždavinyje

Eglė Jasevičiūtė

Karina Stūronaitė

Elzė Voveriūnaitė

Darbo vadovas : Doc., Dr., Viktor Skorniakov

Dėstytoja : Prof., Dr., Jurgita Markevičiūtė

**Vilnius**
**2026**

# Santrauka

Šiame darbe nagrinėjamas muitinės deklaracijose pateikiamų prekių aprašymų ir deklaruotų prekių kodų atitikimas, taikant natūralios kalbos apdorojimo metodus. Deklaruoti ir nomenklatūroje pateikti prekių aprašymai transformuojami į vektorius, leidžiančius įvertinti jų semantinį panašumą ir automatiškai nustatyti galimus neatitikimus. Probleminiams atvejams modelis generuoja labiausiai tikėtinus alternatyvius prekės kodus pagal tekstinį panašumą. Modelio veikimas vertinamas lyginant jo pasiūlymus su rankiniu būdu priskirtais kodais. Taip pat atliekama statistinė analizė, siekiant įvertinti klaidingų deklaracijų mastą žaliojo srauto duomenyse – muitinės patikrą praėjusių deklaracijų aibėje.

**Raktiniai žodžiai:** muitinė, deklaracija, nomenklatūra, prekė, klasifikavimas, TF-IDF, NLP.

# Iliustracijų sąrašas

1 Deklaracijų ir nomenklatūrų aprašymų žodžių skaičiaus stačiakampės diagramos . . . 11
2 Deklaracijose dažniausiai panaudojamų klasių skritulinės diagramos . . . . . 12
3 Deklaracijų prekių aprašymų dažniausių žodžių kamienai . . . . . 13
4 Nomenklatūrų prekių aprašymų dažniausių žodžių kamienai . . . . . 13

# Lentelių sąrašas

1 Originalūs ir darbui paruošti nomenklatūros ir deklaracijų prekių aprašymai . . . . . 10
2 Nomenklatūroje esančių ir deklaracijose panaudotų prekės kodų skaičius pagal metus 12

# Turinys

**Santrauka** . . . . . **2**

**Iliustracijų sąrašas** . . . . . **3**

**Lentelių sąrašas** . . . . . **4**

**Santrumpos** . . . . . **6**

**Įvadas** . . . . . **7**

**1. Literatūros analizė** . . . . . **8**
1.1. Muitinė ir Kombinuotoji nomenklatūra . . . . . 8
1.2. Ankstesni moksliniai tyrimai . . . . . 8
1.3. Darbo metodika . . . . . 9

**2. Duomenų aprašymas ir paruošimo darbui eiga** . . . . . **10**

**3. Pradinė duomenų analizė** . . . . . **11**

**Literatūra ir šaltiniai** . . . . . **14**

**A** priedas. Dirbtinio intelekto naudojimas . . . . . **16**

**B** priedas. Python kodas . . . . . **17**

# Santrumpos

Pasaulio muitinių organizacijos Suderinta prekių aprašymo ir kodavimo sistema – SS.
Europos Sąjungos Kombinuotoji nomenklatūra – KN.

# Įvadas

Muitinės darbuotojai susiduria su labai dideliais prekių srautais, kurių fiziškai patikrinti neįmanoma – rizikingomis nelaikomos prekės yra siunčiamos žaliuoju srautu. Jei deklaruotas prekės kodas neatitinka realaus prekės aprašymo, tokie atvejai gali likti nepastebėti ir sukelti mokesčių netekimo, neteisingo tarifų taikymo ar net ribojamų prekių kontrolės riziką. Todėl aktualu taikyti automatizuotus tekstų analizės metodus, galinčius padėti tiksliau identifikuoti potencialiai problematiškas deklaracijas.

Šio darbo tikslas – pritaikyti ir įvertinti natūralios kalbos apdorojimo modelį, leidžiantį automatiškai muitinės deklaracijose aptikti prekės aprašymo ir prekės kodo neatitikimus, pasiūlyti tikėtinus tinkamus kodus, ir įvertinti tokių neatitikimų paplitimą žaliojo srauto duomenyse.

Šiam tikslui pasiekti keliami uždaviniai:

1. Kiekvienai prekei iš deklaracijų pagal aprašymo panašumą į nomenklatūros aprašymus parinkti tinkamiausią prekės kodą.
2. Prekes, kurių deklaruotas kodas nesutampa su nustatytu tinkamiausiu, pateikti žmogaus rankinei patikrai.
3. Įvertinti modelio efektyvumą, tikrinant, ar deklaruotas kodas yra pateiktas tarp penkių siūlomų prekių kodų.
4. Statistiškai ištirti, kiek muitinės žaliojo srauto duomenyse yra objektų deklaruotų prekės aprašymų ir deklaruotų prekės kodų neatitikimų.

Šiame darbe 1. skyriuje apžvelgiama su darbo tema susijusi literatūra ir kiti šaltiniai, 2. skyriuje aprašomi gauti duomenys ir jų sutvarkymo eiga, o 3. skyriuje atliekama duomenų pradinė analizė. Darbo programinis kodas rašytas `Python` programavimo kalba `Google Colab` aplinkoje `Jupyter Notebook` formatu. Kodas pateiktas B priede.

# 1. Literatūros analizė

## 1.1. Muitinė ir Kombinuotoji nomenklatūra

Lietuvos Respublikos muitinė – valstybinė institucija, prižiūrinti tarptautinę prekybą ir prisidedanti prie visos prekių tiekimo grandinės saugumo užtikrinimo [9]. Prekės, įvežamos, paliekančios ar kitaip kertančios Lietuvos Respublikos sienas, turi būti deklaruojamos nurodant jų prekės kodą ir prekės aprašymą. Labai svarbu yra teisingas prekių tarifinis klasifikavimas – nuo to priklauso, ar teisingai apskaičiuoti muito ir kiti mokesčiai, ar prekė laikytina rizikos objektu. Pasaulio muitinių organizacija yra sudariusi Suderintą prekių aprašymo ir kodavimo sistemą (toliau – SS), pagal kurią yra sudaryta Europos Sąjungos Kombinuotoji nomenklatūra (toliau – KN) [7]. Prekių tarifinis klasifikavimas reiškia prekių priskyrimą konkrečiai KN pozicijai ar subpozicijai [8]. KN kodą sudaro aštuoni skaitmenys: pirmi du nurodo skirsnį, tolimesni skaitmenys detaliau nurodo objekto ypatybes (poziciją, subpoziciją) – šiame darbe nagrinėjamas $73$ KN skirsnis, apimantis gaminius iš juodųjų metalų (iš geležies arba iš plieno) [7], [3]. Prie KN kodo pridedami dar du skaitmenys, nurodantys tarifo priedus [7], tad darbe nagrinėjami dešimt skaitmenų turintys KN kodai, darbe vadinami objekto klase.

## 1.2. Ankstesni moksliniai tyrimai

Anksčiau atliktuose moksliniuose tyrimuose prekių klasifikavimo pagal SS kodus tema nuolat pabrėžiama, kad su muitinės deklaracijose pateiktais prekių aprašymais dažnai būna sudėtinga ir nepatogu dirbti: darbuose pasikartoja tai, kad deklaruoti prekių aprašymai dažnai būna labai trumpi [14], [16], [4] ir ženkliai skiriasi nuo nomenklatūroje pateiktų prekės aprašymų teksto apimtimi ir sudėtingumu [14]. Taip pat buvo pastebėta, kad, kadangi deklaracijas pildo žmonės, jose dažnai lieka nestandartinės kalbos – žodžių, kurie yra nevartotini arba įrašyti su rašybos klaidomis [1], [14], [4]. Be to, panašių objektų deklaracijų aprašymuose dažnai pasikartoja raktiniai žodžiai, kurie atitinka bent kelias galimas objekto klases [1], [14], [16].

Moksliniai straipsniai šia tema siūlo tokius natūralios kalbos paruošimo darbui žingsnius: pasikartojančių įrašų, skyrybos ženklų, dažnai pasikartojančių bereikšmių žodžių (angl. stopwords) šalinimas, teksto perrašymas vien mažosiomis raidėmis [1], [14], [16]. Taip pat siūloma palikti tik žodžių pagrindines formas [1], [14] arba kamienus [18], [4]. Keli darbai taip pat minėjo, kad duomenų paruošimas darbui vis dėlto turėtų būti lankstus ir pritaikytas turimiems duomenims, nes skirtingų prekių aprašymai dažnai būna specifiniai [2], [16].

Norint palyginti dviejų tekstų panašumą (nagrinėjamais atvejais – nomenklatūroje ir deklaracijose randamus prekių aprašymus), darbuose siūloma tekstą vektorizuoti TF-IDF metodu ir tada tarp teksto vektorių skaičiuoti kosinusinį panašumą [5], [18]. Taip pat buvo pasiūlyta be vektorizavimo skaičiuoti, kokia dalis žodžių iš deklaracijos yra ir nomenklatūros aprašyme [6]. Keli darbai lygino paprastus teksto apdirbimo būdus kaip TF-IDF su sudėtingesniais būdais (pvz., *Word2Vec*) ir kalbos modeliais [14], [5].

Darbuose buvo pasiūlyti įvairūs klasifikavimo metodai: K-artimiausių kaimynų (angl. K-nearest neighbour) [1], [14], [18], pasirinkimų medžio (angl. decision tree) [14], atraminių vektorių mašinos (angl. support vector machine) [5], [18] ir atsitiktinio miško (angl. random forest) [4]. Taip pat vienaname darbe buvo tirti didžiausią įtaką metodo klasifikavimo tikslumui darantys K-artimiausių kaimynų ir kitų klasifikavimo metodų hiperparametrai [18]. Keliuose darbuose buvo paminėta, kad siūlant galimas klases vertėtų pateikti kelis alternatyvius variantus, kad sprendimus priimantis žmogus turėtų daugiau konteksto [16], [6].

## 1.3. Darbo metodika

Ruošiant natūralios kalbos tekstinius duomenis darbui (šalinant skyrybos ženklus ir kita) naudojamos reguliariosios išraiškos, rašomos remiantis `Python` paketo `regex` dokumentacija [13]. Taip pat, ypač dirbant su tekstais lietuvių kalba, skirtingas žodžių formas būtina suvienodinti paliekant tik žodžių kamienus – tai atliekama remiantis paketo `SnowballStemmer` dokumentacija [12].

Norint gauti skaičių, rodantį, kiek panašūs yra du tekstai, dažnai reikia juos vektorizuoti. Vienas iš metodų, taikomų natūralios kalbos vektorizavimui, yra TF-IDF [11], [18]. Jo bendra formulė yra tokia:

$$\text{tf-idf}(t,d) = f_{t,d} \cdot \left( \log \left( \frac{1+N}{1+\text{df}(t)} \right) + 1 \right),$$

kur $t$ – konkretus žodis, $d$ – nagrinėjamas dokumentas, $f_{t,d}$ žymi, kiek kartų terminas $t$ pasikartoja dokumente $d$ (termino dažnis), $N$ – bendras visų dokumentų skaičius rinkinyje, o $df(t)$ rodo dokumentų dažnį – kiek dokumentų visame rinkinyje turi žodį $t$.

Tada tarp vektorių galima skaičiuoti panašumą pagal kosinusą, kuris įgyja reikšmes $[0,1]$, kur $0$ – tekstų vektoriai visiškai nepanašūs, $1$ – tekstų vektoriai sutampa. Kosinusinio panašumo formulė užrašoma taip [10]:

$$cos(A,B) = \frac{< A, B >}{\|A\| \cdot \|B\|},$$

kur $A,B$ – lyginami tekstų vektoriai, $< A, B >$ – vektorių skaliarinė sandauga, o $\|A\| \cdot \|B\|$ – vektorių ilgių sandauga.

Dviejų tekstų panašumą galima įvertinti ir be vektorizavimo [6] pagal tokią formulę:

$$S(A,B) = \frac{\text{bendrų žodžių skaičius}}{\text{žodžių skaičius tekste } A},$$

kur $A,B$ – originalūs ne vektorizuoti tekstai, šiuo atveju $A$ yra deklaracijos prekės aprašymas, o $B$ – nomenklatūros prekės aprašymas.

# 2. Duomenų aprašymas ir paruošimo darbui eiga

Iš Lietuvos Respublikos muitinės buvo gauta pilna 2023 m. visų prekių hierarchinė Kombinuotoji nomenklatūra, 2024 bei 2025 m. įsigalioję nomenklatūros pakeitimai, taip pat visų 2023 m. bei nuo 2024 m. kovo mėn. iki 2026 m. kovo mėn. laikotarpio muitinei pateiktų deklaracijų apie $73$ skirsnio prekes duomenys.

Rankiniu būdu buvo padaryti trys atskiri KN failai su pakeitimais 2024 bei 2025 m. ir tada suplokštinti, kad vietoje hierarchinės struktūros liktų tik tie kodai, kurie realiai yra naudojami deklaracijose. Plokštinant hierarchinės struktūros duomenis išsaugoti visų platesnių vienetų, išskyrus skirsnio, kuris pasikartoja visoms klasėms, aprašymai, tad kiekvienai klasei pateiktas išsamus aprašymas. Deklaracijų duomenys buvo sujungti į vieną failą ir pašalinti 2026 m. duomenys, nes muitinė 2026 m. įsigaliojusių KN pakeitimų nepateikė.

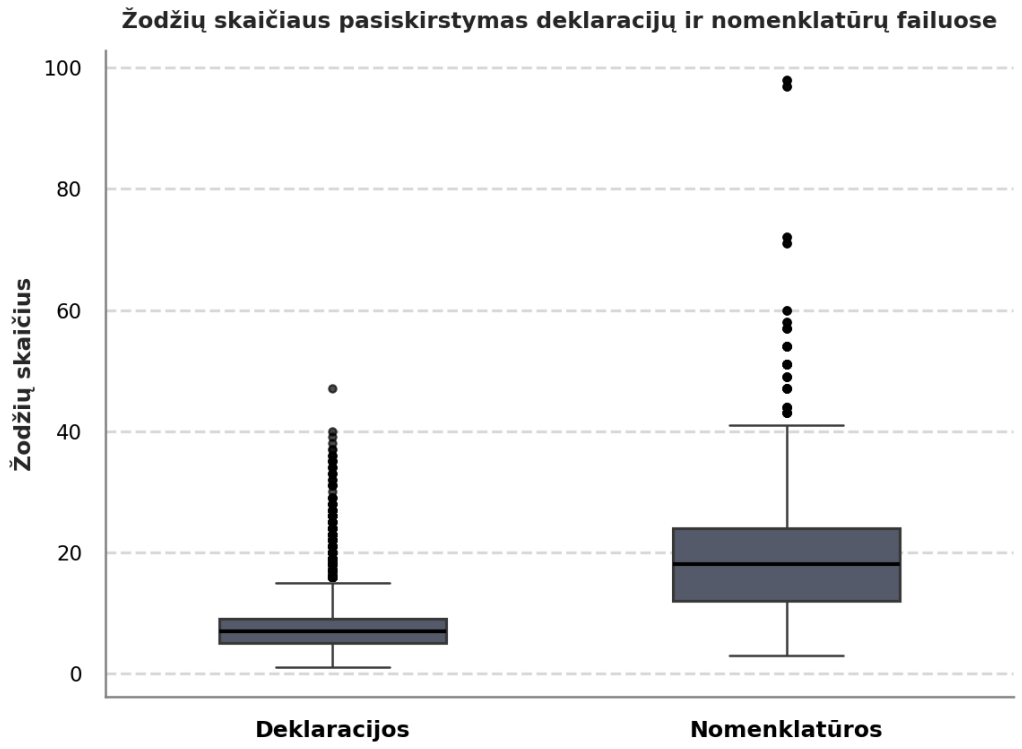
Toliau tiek deklaracijų, tiek nomenklatūros failuose esantys prekių aprašymai buvo parengti darbui pagal darbo su natūralia kalba principus: visas tekstas perrašytas tik mažosiomis raidėmis, neaktualūs skyrybos ženklai ir dažni, tačiau reikšmės neturintys, žodžiai pašalinti. Lietuviški simboliai („ą", „č" ir kt.) pakeisti jų nelietuviškais atitikmenimis („a", „c" ir kt.). Taip pat palikti tik žodžių kamienai, tad iš, pavyzdžiui, žodžių „varžtas", „varžtai", „varžtų" lieka identiški kamienai „varžt". Skaičių su matavimo vienetais reikšmė išsaugota, kadangi metalo gaminiai kartais klasifikuojami pagal jų išmatavimus. Išsaugotuose skaičiuose dešimtainės dalies atskyrimo simboliai suvienodinti į taškus. Pavyzdiniai originalūs ir paruošti darbui prekių aprašymai iš nomenklatūros ir deklaracijų duomenų aibių pateikti 1 lentelėje. Galiausiai iš deklaracijų duomenų pašalintos besidubliuojančios eilutės, kuriose sutampa deklaruotas prekės kodas ir aprašymas, tad iš $195482$ lieka $40909$ eilutės. Prieš duomenų sutvarkymą deklaracijose unikalūs žodžiai sudarė vos $2{,}67$ % visų žodžių, o pašalinus besidubliuojančias eilutes unikalūs žodžiai sudarė $8{,}93$ % – procentinė dalis smarkiai padidėjo, nors ir palikti tik žodžių kamienai.

***1 lentelė.*** *Originalūs ir darbui paruošti nomenklatūros ir deklaracijų prekių aprašymai*

| Prekės aprašymo vieta | Originalus aprašymas | Sutvarkytas aprašymas |
|---|---|---|
| Nomenklatūra | Kiti vamzdžiai ir vamzdeliai (pavyzdžiui, suvirinti, sukniedyti arba sujungti panašiu būdu), apskrito vidinio ir išorinio skerspjūvio, kurių išorinis skersmuo didesnis kaip 406,4 mm, iš geležies arba iš plieno | kiti vamzdz vamzdel pavyzdz suvir suknied sujung pana bud apskrit vidin isorin skerspjuv isorin skersm did 406.4_mm gelez plien |
| Deklaracijos | PLIENINIAI LYNAI DENGTI ZN 0,8MM, 1.0MM, 1,5MM, DENGTI PVC 1/2MM RITINIUOSE. NETINKA KĖLIMUI | plienin lyn deng zn 0.8_mm 1.0_mm 1.5_mm deng pvc 1/2_mm ritin netink kelim |

# 3. Pradinė duomenų analizė

Pirmiausia buvo palyginti deklaracijų ir nomenklatūros prekių aprašymuose esančių žodžių skaičiai (žr. 1 pav.). Skirtingų metų nomenklatūrų aprašymų žodžių skaičiaus vidurkiai ir standartiniai nuokrypiai tarpusavyje skiriasi per šimtąją dalį, tad grafike pavaizduotas vidutinis visų trijų nomenklatūros variantų aprašymų žodžių skaičius. Iš stačiakampių diagramų matosi, kad aiškiai skiriasi žodžių skaičius – deklaracijų aprašymuose esančių žodžių vidurkis yra $\sim 7$, su stand. nuokrypiu, lygiu $3{,}39$, o visų trijų nomenklatūrų – atitinkamai $\sim 19{,}6$ ir $\sim 11{,}5$. Atlikti *Mann-Whitney U* statistiniai testai [15] su pasikliovimo lygmeniu $95$ %, lyginant žodžių skaičių viename aprašyme (t.y. viename objekte) tarp deklaracijų ir kiekvienos iš trijų nomenklatūrų (nepriklausomos imtys). Nulinė hipotezė teigia, kad žodžių skaičiaus viename aprašyme pasiskirstymai tarp lyginamų grupių nesiskiria, o alternatyvioji – kad deklaracijų aprašymai yra trumpesni. Visais atvejais gautos p reikšmės buvo mažesnės už $0{,}05$, todėl nulinė hipotezė atmetama. Tai rodo, kad deklaracijų aprašymuose žodžių skaičius viename įraše yra statistiškai reikšmingai mažesnis nei visų trijų nomenklatūrų aprašymuose.



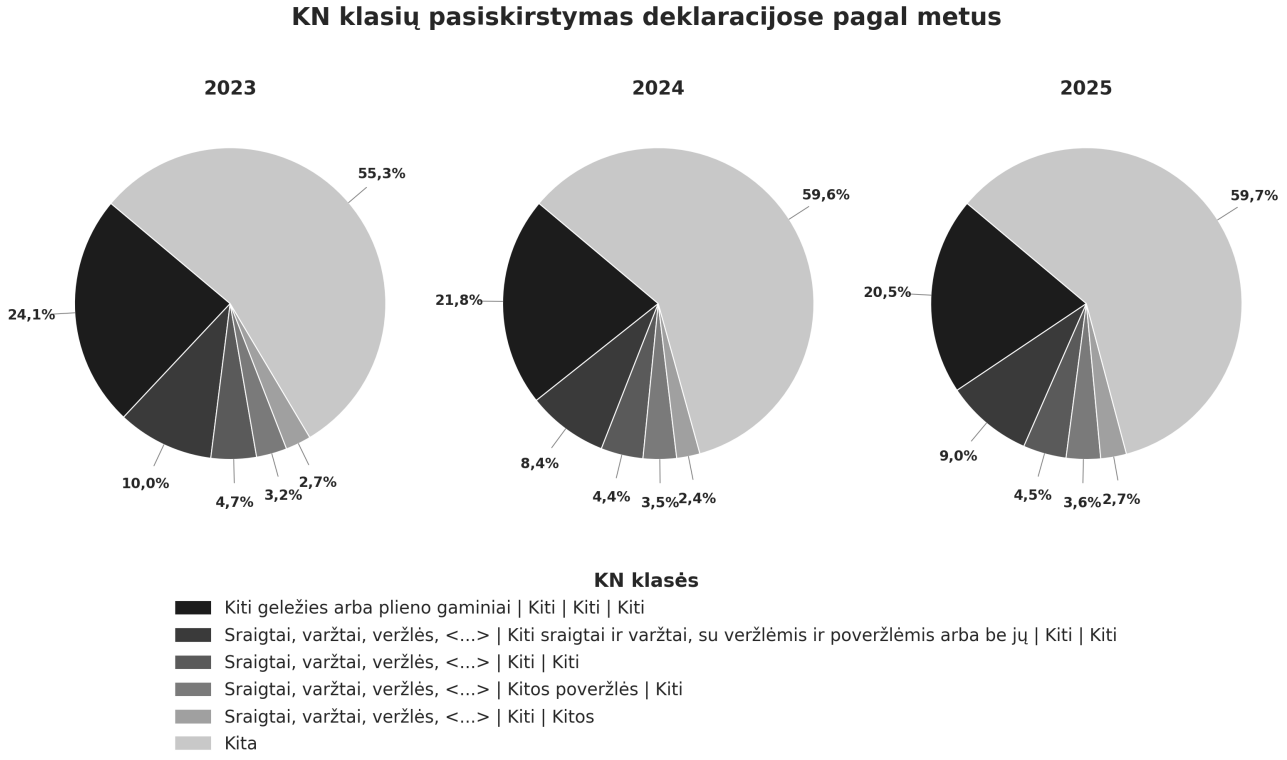

***1 pav.*** *Deklaracijų ir nomenklatūrų aprašymų žodžių skaičiaus stačiakampės diagramos*

Taip pat patikrinta, kiek klasių kodų yra kiekvienų metų nomenklatūroje ir kiek iš jų tais metais buvo panaudota deklaracijose – rezultatai pavaizduoti 2 lentelėje. Pagal lentelę matyti, kad visais metais mažiau nei pusė nomenklatūros kodų yra panaudojami deklaracijose.

***2 lentelė.*** *Nomenklatūroje esančių ir deklaracijose panaudotų prekės kodų skaičius pagal metus*

| Metai | Kodų sk. nomenklatūroje | Kodų sk. deklaracijose | Procentinė dalis |
|---|---|---|---|
| 2023 | 599 | 268 | 44,7 % |
| 2024 | 601 | 232 | 38,6 % |
| 2025 | 602 | 238 | 39,5 % |

Nagrinėjant prekių kodų dažnį dar buvo patikrinta, kurios klasės deklaracijose yra įrašomos dažniausiai – skritulinės diagramos kiekvieniems metams pavaizduotos 2 paveiksle. Matyti, kad visais metais dažniausių klasių išsidėstymas sutampa, tik skiriasi jų dažniai. Vos $5$ dažniausios klasės iš $\sim$ $600$ įmanomų klasių kiekvienais metais sudaro $\sim$ $40$ % visų užpildytų deklaracijų. Dažniausiai deklaracijose naudojamos yra nespecifinės klasės, kur poskyriai yra „kiti", „kitos" ir pan.




***2 pav.*** *Deklaracijose dažniausiai panaudojamų klasių skritulinės diagramos*

Galiausiai deklaracijų ir bendrai nomenklatūrų prekių aprašymams buvo nubraižyti žodžių debesų (angl. wordcloud) grafikai (žr. 3 ir 4 pav.), kuriuose vaizduojami 100 dažniausiai prekių aprašymuose pasitaikančių žodžių kamienų. Matyti, kad žodžių debesys labai skiriasi ir dažniausi žodžiai nesutampa, tad pasiekti aukštą klasifikavimo tikslumą turbūt bus sudėtinga.




***3 pav.*** *Deklaracijų prekių aprašymų dažniausių žodžių kamienai*




***4 pav.*** *Nomenklatūrų prekių aprašymų dažniausių žodžių kamienai*

# Literatūra ir šaltiniai


[1] F. Altaheri, K. Shaalan. „Exploring machine learning models to predict harmonized system code“. Iš: *European, Mediterranean, and Middle Eastern Conference on Information Systems*. Springer. 2019, puslapiai 291–303.

[2] O. Cherednichenko, O. Ivashchenko, M. Lincényi, M. Kováč. „Information technology for intellectual analysis of item descriptions in e-commerce“. Iš: *Entrepreneurship and Sustainability Issues* 11.1 (2023), puslapis 178.

[3] EUR-Lex. *Europos Sąjungos kombinuotosios nomenklatūros paaiškinimai*. URL: https://eur-lex.europa.eu/legal-content/LT/TXT/?uri=celex%3A52019XC0329%2802%29 (žiūrėta 2026-03-08).

[4] P. Harsani, A. Suhendra, L. Wulandari, W. C. Wibowo. „A study using machine learning with Ngram model in harmonized system classification“. Iš: *J. Adv. Res. Dyn. Control Syst* 12.6 (2020), puslapiai 145–153.

[5] S. M. Krupa, Y. P. Kryvenchuk. „Models of semantic analysis of product descriptions for automatic determination of customs codes“. Iš: *Applied Aspects of Information Technology* 9.1 (2026), puslapiai 76–85.

[6] E. Lee, S. Kim, S. Kim, S. Park ir kiti. „Classification of goods using text descriptions with sentences retrieval“. Iš: *arXiv preprint arXiv:2111.01663* (2021).

[7] L. R. muitinė. *Bendrieji prekių tarifinio klasifikavimo principai*. URL: https://muitine.lrv.lt/public/canonical/1746427701/269/1_Bendrieji%20tarifinio%20klasifikavimo%20principai_250429.pdf (žiūrėta 2026-03-08).

[8] L. R. muitinė. *Prekių tarifinis klasifikavimas*. URL: https://muitine.lrv.lt/lt/verslui/muitai-ir-mokesciai/prekiu-tarifinis-klasifikavimas/ (žiūrėta 2026-03-06).

[9] L. R. muitinė. *Vizija, misija, vertybės*. URL: https://muitine.lrv.lt/lt/apie-mus/vizija-misija-vertybes/ (žiūrėta 2026-03-01).

[10] Paketo *scikit-learn* dokumentacija. *Funkcijos cosine*$_s imilaritydokumentacija$. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html.

[11] Paketo *scikit-learn* dokumentacija. *Funkcijos TfidfVectorizer dokumentacija*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (žiūrėta 2026-04-02).

[12] PyPI. *Paketo SnowballStemmer dokumentacija*. URL: https://pypi.org/project/snowballstemmer/ (žiūrėta 2026-04-02).

[13] *Python* dokumentacija. *Paketo regex dokumentacija*. URL: https://docs.python.org/3/library/re.html#id1 (žiūrėta 2026-04-02).

[14] E. D. B. Sasana, D. Siahaan, D. Purwitasari. „Towards Better HS Code Prediction: A Comparative Study of Machine Learning and NLP Approaches“. Iš: *2025 International Conference on Smart Computing, IoT and Machine Learning (SIML)*. IEEE. 2025, puslapiai 1–7.

[15] SciPy. *Mann-Whitney U testo funkcijos dokumentacija*. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.mannwhitneyu.html.

[16] S. Stassin, O. Amel, S. Mahmoudi, X. Siebert. „Similarity versus Supervision: Best Approaches for HS Code Prediction“. Iš: *ESANN*. 2023.

[17] VU MIF. *Latex rašto darbo šablonas lietuvių kalba*. URL: https://mif.vu.lt/lt3/dokumentai/sablonai/LaTex%20MIF%20RD%20Sablonas.zip.

[18] F. R. Zagatti, G. Y. Shimizu, D. Lucrédio, H. M. Caseli. „Investigating the relationship between text vectorization cosine similarity and classification performance“. Iš: *IEEE Access* 13 (2025), puslapiai 137348–137363.

# **A** priedas. Dirbtinio intelekto naudojimas

Dirbtinio intelekto modelis „ChatGPT“ buvo naudotas kaip pagalbinis įrankis LATEX, Python kodo rašymui.

# B priedas. Python kodas

```
import pandas as pd
import re
import numpy as np
import snowballstemmer
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from collections import Counter
import random
```

```
def clean_goods_code(code):
    if pd.isna(code):
        return None
    code_str = str(code).replace(" ", "")
    return code_str[:10]

def get_indent_depth(indent):
    if pd.isna(indent) or str(indent).strip() == '':
        return 0
    return str(indent).count('-')

print("Sukurti failai: ")
def process_kn_file(file_path, output_path):
    df = pd.read_excel(file_path)
    df.columns = df.columns.str.strip()
    df["Goods code"] = df["Goods code"].apply(clean_goods_code)
    df["indent_depth"] = df["Indent"].apply(get_indent_depth)

    df_sorted = df.sort_values(by=["Goods code", "indent_depth"], ascending=[True,
    ↪  True])
    df_leaf = df_sorted.groupby("Goods code").last().reset_index()

    df_hier = df.sort_values(by=["Goods code", "Hier. Pos.",
    ↪  "indent_depth"]).reset_index(drop=True)

    results = []
    stack = []

    for i in range(len(df_hier)):
        row = df_hier.iloc[i]
```

```
        indent = int(row["Hier. Pos."])
        desc = str(row["Description"]).strip()
        code = row["Goods code"]

        while stack and stack[-1][0] >= indent:
            stack.pop()
        stack.append((indent, desc))

        max_depth = df_hier[df_hier["Goods code"] == code]["indent_depth"].max()
        if row["indent_depth"] == max_depth:
            full_description = " | ".join([d for _, d in stack])
            results.append({
                "Goods code": code,
                "Full description": full_description,
                "Start date": row.get("Start date"),
                "End date": row.get("End date"),
                "Language": row.get("Language")
            })

    result_df = pd.DataFrame(results)
    result_df = result_df.drop_duplicates(subset=['Goods code'], keep='last')
    result_df.to_csv(output_path, index=False, encoding="utf-8-sig")
    print(f"{output_path}")

files = {
    "KN 2023.xlsx": "KN_2023_flat.csv",
    "KN 2024_pilnas.xlsx": "KN_2024_flat.csv",
    "KN 2025_pilnas.xlsx": "KN_2025_flat.csv"
}

for input_file, output_file in files.items():
    process_kn_file(input_file, output_file)
```

```
def normalize_file(file_path):
    df = pd.read_excel(file_path)
    df.columns = df.columns.str.strip()
    col_map = {}

    for col in df.columns:
        if "018." in col or "020." in col:
            col_map[col] = "Data"
        elif "071." in col or "063." in col:
            col_map[col] = "Kodas"
        elif "085." in col or "072." in col:
```

```
            col_map[col] = "Aprašymas"

    df = df.rename(columns=col_map)

    df = df[["Data", "Kodas", "Aprašymas"]].copy()

    df["Kodas"] = (
        df["Kodas"]
        .astype(str)
        .str.replace(" ", "", regex=False)
        .str[:10]
    )

    return df

files = [
    "Importo_prekes_lygio_2023_I_ketv_uzkod.xlsx",
    "Importo_prekes_lygio_2023_II_ketv uzkod.xlsx",
    "Importo_prekes_lygio_2023_III_ketv_uzkod.xlsx",
    "Importo_prekes_lygio_2023_IV_ketv_uzkod.xlsx",
    "Importo_prekes_lygio_20240311-20260315_uzkod.xlsx"
]

all_data = []

for file in files:
    df = normalize_file(file)
    all_data.append(df)

combined_df = pd.concat(all_data, ignore_index=True)

combined_df["Data"] = combined_df["Data"].astype(str)
combined_df = combined_df[~combined_df["Data"].str.startswith("2026")]

combined_df.to_csv("combined_imports_cleaned.csv", index=False, encoding="utf-8-sig")

print("Sukurtas failas:")
print("combined_imports_cleaned.csv")
```

```
import re
import pandas as pd
import snowballstemmer

kn_files = ["KN_2023_flat.csv", "KN_2024_flat.csv", "KN_2025_flat.csv"]
```

```
combined_file = "combined_imports_cleaned.csv"

stemmer = snowballstemmer.stemmer('lithuanian')

lt_letter_map = str.maketrans({
    'ą': 'a', 'č': 'c', 'ę': 'e', 'e': 'e',
    'i': 'i', 'š': 's', 'u': 'u', 'u': 'u', 'ž': 'z',
})

raw_stopwords = [
    'ir', 'iš', 'su', 'be', 'ar', 'bei', 'bet', 'tai', 'kad', 'jei',
    'per', 'prie', 'ant', 'už', 'iki', 'nuo', 'i', 'po', 'tarp', 'apie',
    'arba', 'nei', 'kaip', 'tik', 'dar', 'jau', 'ne', 'taip', 'kurie',
    'kurio', 'kurios', 'kuriu', 'kuri', 'kuriam', 'kuriems', 'kuriuo',
    'kuriomis', 'kuriose', 'kitas', 'kiti', 'kito', 'kitu', 'kita',
    'naudojamas', 'naudojami', 'naudojama', 'naudoti', 'skirtas',
    'skirti', 'skirta', 'skirtu', 'tipo', 'rušies', 'rušiu'
]

normalized_stopwords = [w.lower().translate(lt_letter_map) for w in raw_stopwords]
stop_words = set(stemmer.stemWords(normalized_stopwords))

def preprocess_text(text):
    if pd.isna(text):
        return ""

    text = text.lower()
    text = text.translate(lt_letter_map)
    text = re.sub(r'(\d+),(\d+)', r'\1.\2', text)
    text = re.sub(r'(\d+\.\d+)', lambda m: m.group(1).replace('.', 'XDOTX'), text)
    text = re.sub(r'[^\w\s\-_/]', ' ', text)
    text = text.replace('XDOTX', '.')
    text = re.sub(r'(\d+(?:[./]\d+)*)\s*(mm|cm|m|kg|g|l|ml|t|oz|in|ft)\b', r'\1_\2',
    ↪  text)
    text = re.sub(r'\s+', ' ', text).strip()

    tokens = text.split()
    tokens = stemmer.stemWords(tokens)
    tokens = [
        t for t in tokens
        if t not in stop_words
        and len(t) > 1
        and re.search(r'[a-z0-9]', t)
```

```
    ]

    return " ".join(tokens)

print("Sukurti failai:")
for f in kn_files:
    year = re.search(r'KN_(\d{4})_flat', f).group(1)
    df = pd.read_csv(f)

    df['cleaned_description'] = df['Full description'].apply(preprocess_text)

    output_file = f"KN_{year}_cleaned.csv"
    df.to_csv(output_file, index=False,
              columns=['Goods code', 'Full description', 'cleaned_description'])

    print(f"{output_file}")

combined_df = pd.read_csv(combined_file)

combined_df['cleaned_description'] = combined_df['Aprašymas'].apply(preprocess_text)

before = len(combined_df)
combined_df = combined_df.drop_duplicates(subset=['Kodas', 'cleaned_description'])
after = len(combined_df)

combined_df.to_csv(
    "combined_imports_cleaned_prepared.csv",
    index=False,
    columns=['Data', 'Kodas', 'Aprašymas', 'cleaned_description']
)

print("combined_imports_cleaned_prepared.csv")
print(f"\nIš deklaraciju pašalinta {before - after} besidubliuojančiu eilučiu ({before}
↪    {after} eilučiu)")
print("\nUnikaliu žodžiu skaičius deklracijos sutvarkytame ir pradiniame failuose:")

def get_words(series):
    return [w for text in series.dropna() for w in str(text).split()]

decl_words_clean = get_words(combined_df['cleaned_description'])
raw_decl_df = pd.read_csv("combined_imports_cleaned.csv")
decl_words_raw = get_words(raw_decl_df['Aprašymas'])

```

```
decl_clean_unique = len(set(decl_words_clean))
decl_raw_unique = len(set(decl_words_raw))
decl_clean_total = len(decl_words_clean)
decl_raw_total = len(decl_words_raw)

print(f"  pradinis   unikaliu: {decl_raw_unique:>7}  ({decl_raw_unique/decl_raw_total*100:.2f}%) | iš viso: {decl_raw_total:>9}")
print(f"  cleaned    unikaliu: {decl_clean_unique:>7}  ({decl_clean_unique/decl_clean_total*100:.2f}%) | iš viso: {decl_clean_total:>9}")
```

```
kn_files = {
    2023: "KN_2023_cleaned.csv",
    2024: "KN_2024_cleaned.csv",
    2025: "KN_2025_cleaned.csv"
}
kn_flat_files = {
    2023: "KN_2023_flat.csv",
    2024: "KN_2024_flat.csv",
    2025: "KN_2025_flat.csv"
}
kn_raw_files = {
    2023: "KN 2023.xlsx",
    2024: "KN 2024_pilnas.xlsx",
    2025: "KN 2025_pilnas.xlsx"
}
combined_file = "combined_imports_cleaned_prepared.csv"

combined_df = pd.read_csv(combined_file)
kn_dfs = {year: pd.read_csv(path) for year, path in kn_files.items()}
kn_flat_dfs = {year: pd.read_csv(path) for year, path in kn_flat_files.items()}

combined_df['word_count'] = combined_df['cleaned_description'].str.split().str.len()
wc_groups = {'Deklaracijos': combined_df['word_count'].dropna()}

all_kn_wc = []

for year, df in kn_dfs.items():
    df['word_count'] = df['cleaned_description'].str.split().str.len()
    wc = df['word_count'].dropna()
    wc_groups[f'KN {year}'] = wc
    all_kn_wc.extend(wc)

all_kn_wc = pd.Series(all_kn_wc)

```

```
print("Žodžiu skaičiaus statistika")

for name, wc in wc_groups.items():
    print(f"{name:<15}  vidurkis={wc.mean():.2f}  std={wc.std():.2f}  n={len(wc)}")

decl_wc = wc_groups['Deklaracijos']

print("\nMann Whitney U testas (Deklaracijos vs KN):")

for year in kn_dfs.keys():
    kn_wc = wc_groups[f'KN {year}']
    u, p = stats.mannwhitneyu(decl_wc, kn_wc, alternative='less')
    print(f"\nDeklaracijos vs KN {year}")
    print(f"U = {u:.4f}, p = {p:.2e}")

print("\n")

plot_df = pd.concat([
    pd.DataFrame({'word_count': decl_wc, 'grup ': 'Deklaracijos'}),
    pd.DataFrame({'word_count': all_kn_wc, 'grup ': 'Nomenklat ros'})
], ignore_index=True)

sns.set_theme(style="whitegrid", context="talk")

fig, ax = plt.subplots(figsize=(8, 6), dpi=150)

palette = {
    "Deklaracijos": "#50586C",
    "Nomenklaturos": "#50586C"
}

sns.boxplot(
    data=plot_df,
    x='grup ',
    y='word_count',
    hue='grup ',
    palette=palette,
    width=0.5,
    showfliers=True,
    dodge=False,
    legend=False,
    flierprops={
        "marker": "o",
```

```
        "markersize": 4,
        "markerfacecolor": "black",
        "markeredgecolor": "black",
        "alpha": 0.7
    },
    linewidth=1.5,
    medianprops={
        "color": "black",
        "linewidth": 2
    },
    whiskerprops={"linewidth": 1.2},
    capprops={"linewidth": 1.2},
    ax=ax
)

ax.set_title(
    "Žodžiu skaičiaus pasiskirstymas deklaraciju ir nomenklaturu failuose",
    fontsize=12,
    fontweight='bold',
    pad=12
)

ax.set_xlabel("")
ax.set_ylabel("Žodžiu skaičius", fontsize=12, fontweight='bold')

ax.tick_params(axis='x', labelsize=12, colors='black')
ax.tick_params(axis='y', labelsize=11, colors='black')

for label in ax.get_xticklabels():
    label.set_fontweight('bold')
    label.set_color('black')

ax.grid(True, axis='y', linestyle='--', alpha=0.3, color='gray')
ax.grid(False, axis='x')

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

ax.spines['left'].set_color("gray")
ax.spines['bottom'].set_color("gray")

ax.spines['left'].set_linewidth(1.2)
ax.spines['bottom'].set_linewidth(1.2)
```

```
121
122 plt.tight_layout()
123 plt.show()
124
125 print("\n")
126
127 print("Panaudoti KN kodai deklaracijose (pagal metus)")
128 combined_df['year'] = pd.to_datetime(combined_df['Data'], errors='coerce').dt.year
129
130 for year, df in kn_dfs.items():
131     kn_codes = set(df['Goods code'].dropna().astype(str).str.replace(" ", ""))
132     decl_year = combined_df[combined_df['year'] == year]
133     used_codes = set(decl_year['Kodas'].dropna().astype(str)) & kn_codes
134     print(f"{year}: {len(used_codes)} panaudot  / {len(kn_codes)} KN kod  "
135           f"({100 * len(used_codes) / len(kn_codes):.1f}%)")
136
137 print("\n")
138
139 print("Dažniausiai naudojamos KN klases deklaracijose (pagal metus)\n")
140
141 for year, df in kn_flat_dfs.items():
142     kn_codes = df[['Goods code', 'Full description']].dropna().copy()
143     kn_codes['code_clean'] = kn_codes['Goods code'].astype(str).str.replace(" ", "")
144
145     decl_year = combined_df[combined_df['year'] == year].copy()
146     decl_year['code_clean'] = decl_year['Kodas'].dropna().astype(str)
147
148     merged = decl_year.merge(kn_codes, left_on='code_clean', right_on='code_clean',
      ↪  how='inner')
149
150     top5 = (
151         merged.groupby(['code_clean', 'Full description'])
152         .size()
153         .sort_values(ascending=False)
154         .head(5)
155         .reset_index()
156     )
157     top5.columns = ['Kodas', 'Aprašymas', 'Kiekis']
158
159     print(f"--- {year} ---")
160     for i, row in top5.iterrows():
161         print(f"  {i+1}. [{row['Kodas']}] {row['Aprašymas']}  ({row['Kiekis']})")
162     print()
```

```
163
164 print("\n")
165
166 def shorten_label(label, all_labels, prefix_max=30):
167     if label == 'Kita':
168         return label
169
170     parts = [p.strip() for p in label.split('|')]
171
172     same_group = [
173         l for l in all_labels
174         if l != label and l != 'Kita'
175         and l.split('|')[0].strip() == parts[0]
176     ]
177
178     if not same_group:
179         if len(parts) == 1:
180             return parts[0]
181         return parts[0] + ' | ' + ' | '.join(parts[1:])
182
183     all_parts = [[p.strip() for p in l.split('|')] for l in same_group]
184
185     common_depth = 0
186     for depth in range(len(parts)):
187         if all(depth < len(op) and op[depth] == parts[depth] for op in all_parts):
188             common_depth = depth + 1
189         else:
190             break
191
192     if common_depth == 0 or common_depth >= len(parts):
193         return parts[0]
194
195     first_words = ' '.join(parts[0].split()[:3])
196     unique_parts = ' | '.join(parts[common_depth:])
197     return f"{first_words} <...> | {unique_parts}"
198
199
200 top_n = 5
201 all_counts = {}
202 all_labels_ordered = []
203
204 for i, (year, df) in enumerate(kn_flat_dfs.items()):
205     kn_codes = df[['Goods code', 'Full description']].dropna().copy()
```

```
    kn_codes['code_clean'] = kn_codes['Goods code'].astype(str).str.replace(" ", "")

    decl_year = combined_df[combined_df['year'] == year].copy()
    decl_year['code_clean'] = decl_year['Kodas'].dropna().astype(str)

    merged = decl_year.merge(kn_codes, on='code_clean', how='inner')

    counts = (
        merged.groupby(['code_clean', 'Full description'])
        .size()
        .sort_values(ascending=False)
    )
    counts.index = [desc for _, desc in counts.index]

    top = counts.head(top_n)
    other_val = counts.iloc[top_n:].sum()
    counts_plot = pd.concat([top, pd.Series({'Kita': other_val})])
    all_counts[year] = counts_plot

    if i == 0:
        all_labels_ordered = list(counts_plot.index)

palette_colors = ["#1c1c1c", "#3a3a3a", "#5a5a5a", "#7a7a7a", "#a0a0a0", "#c8c8c8"]
color_map = {label: palette_colors[i] for i, label in enumerate(all_labels_ordered)}

non_kita = [l for l in all_labels_ordered if l != 'Kita']
legend_labels_display = [shorten_label(l, non_kita) for l in all_labels_ordered]

fig, axes = plt.subplots(1, 3, figsize=(21, 10), dpi=150)

for ax, (year, counts_plot) in zip(axes, all_counts.items()):
    ordered_labels = [l for l in all_labels_ordered if l in counts_plot.index]
    ordered_values = [counts_plot[l] for l in ordered_labels]
    ordered_colors = [color_map[l] for l in ordered_labels]

    wedges, _ = ax.pie(
        ordered_values,
        labels=None,
        colors=ordered_colors,
        startangle=140,
        wedgeprops={"linewidth": 1.2, "edgecolor": "white"}
    )

```

```
    total = sum(ordered_values)

    for wedge, value in zip(wedges, ordered_values):
        angle = (wedge.theta2 + wedge.theta1) / 2.0
        x = np.cos(np.deg2rad(angle))
        y = np.sin(np.deg2rad(angle))

        percent = 100 * value / total

        ax.plot([x, 1.15 * x], [y, 1.15 * y], color='gray', linewidth=1)

        ax.text(
            1.28 * x,
            1.28 * y,
            f"{percent:.1f}%".replace('.', ','),
            ha='center',
            va='center',
            fontsize=16,
            fontweight='bold'
        )

    ax.set_title(f"{year}", fontsize=22, fontweight='bold', pad=16)

legend_handles = [
    plt.Rectangle((0, 0), 1, 1, color=color_map[l])
    for l in all_labels_ordered
]

legend = fig.legend(
    legend_handles,
    legend_labels_display,
    loc="lower center",
    bbox_to_anchor=(0.5, -0.2),
    fontsize=20,
    frameon=False,
    ncol=1,
    title="KN klases",
    title_fontsize=22
)

legend.get_title().set_fontweight('bold')

fig.suptitle(
```

```
    "KN klasiu pasiskirstymas deklaracijose pagal metus",
    fontsize=28,
    fontweight='bold',
    y=1.02
)

plt.tight_layout()
plt.subplots_adjust(wspace=0.1, bottom=0.18)

plt.show()

print("\n")

def get_raw_words(series):
    return [w for text in series.dropna() for w in str(text).split() if len(w) > 1]

def dark_grey_color(*args, **kwargs):
    val = random.randint(0, 100)
    return f"rgb({val}, {val}, {val})"

def make_wordcloud(words, title):
    freq = Counter(words)

    wc = WordCloud(
        width=900,
        height=450,
        background_color='white',
        max_words=100,
        color_func=dark_grey_color
    ).generate_from_frequencies(freq)

    plt.figure(figsize=(12, 5))
    plt.imshow(wc, interpolation='bilinear')
    plt.axis('off')
    plt.title(title, fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

decl_words_wc = get_raw_words(combined_df['cleaned_description'])

all_kn_words_wc = []
for year, df in kn_dfs.items():
    all_kn_words_wc.extend(get_raw_words(df['cleaned_description']))
```

```
335
336 make_wordcloud(decl_words_wc, "Deklaraciju žodžiu debesis")
337 print("\n")
338 make_wordcloud(all_kn_words_wc, "Nomenklaturu žodžiu debesis (KN 2023 2025)")
```